

Aufgabenblatt 5

Dynamische Programmierung

Wichtige Ankündigungen

- Es gibt drei Aufgabentypen:
 - **Tutorium** → Besprechung im Tutorium
 - **Hausaufgabe** → Eigenarbeit mit Hilfe in Rechnerübungen
 - **Klausurvorbereitung** → Optionale Klausurübung

Abgabe (bis 16.06.2025 23:59 Uhr)

Die folgenden Dateien werden für die Bepunktung berücksichtigt:

Geforderte Dateien:

Blatt05/src/RowOfBowls.java Aufgabe 2.1 bis 2.3

Blatt05/src/Genomics.java Aufgabe 3.1 (Übung ohne Punkte, aber mit Tests)

Als Abgabe wird jeweils nur die letzte Version im main branch in git gewertet.

Aufgabe 1: Palindrom-Teilfolge (Tutorium)

Ein *Palindrom* ist eine Zeichenfolge die rückwärts gelesen dieselbe Folge ergibt wie vorwärts gelesen. Das Wort RENTNER ist also ein Palindrom. Eine *Palindrom-Teilfolge* ist eine Teilfolge einer Zeichenkette, die ein Palindrom ist. Dabei muss die Teilfolge nicht an einem Stück in der gegebenen Zeichenkette vorkommen.

Beispiel: DATENSTRUKTUREN enthält u.a. die Palindrom-Teilfolgen NRUKURN und ERUTURE. Auch UKU, NN und jeder einzelne Buchstabe sind (kurze) Palindrom-Teilfolgen der Zeichenkette DATENSTRUKTUREN.

Es soll ein Algorithmus entwickelt werden, der zu einer gegebenen Zeichenkette str die Länge der längsten Palindrom Teilfolge bestimmt.

1.1 Welche Laufzeit hat der *brute-force* Ansatz (alle Möglichkeiten durchprobieren)?

Lösung

Für alle 2^n Teilfolgen (2^n)
 prüfe, ob Palindrom ($n/2$)
→ $O(n \cdot 2^n)$
(Bemerkung: $O(2^n)$ vermutlich auch möglich)

1.2 Geben Sie eine rekursive Definition der $OPT(i, j)$ an, die als Grundlage für dynamisches Programmieren verwendet werden kann.

Lösung

Die Idee: prüfe von Außen nach Innen, ob Palindrom-Teilfolgen gefunden werden. Setze also zunächst $i = 0$ und $j = n - 1$. Halte die gefundenen Werte in einer Tabelle fest. Es bietet sich an, zunächst die Grenzfälle zu betrachten, also Eingaben mit maximal 4 Zeichen.

$$OPT(i, j) = \begin{cases} 1 & \text{falls } i = j \\ 2 & \text{falls } str[i] = str[j] \text{ und } i + 1 = j \\ OPT(i + 1, j - 1) + 2 & \text{falls } str[i] = str[j] \\ \max(OPT(i, j - 1), OPT(i + 1, j)) & \text{sonst} \end{cases}$$

Alternativ kann man den 2. Fall weglassen und $OPT(i, j) = 0$ für $i = j + 1$ hinzufügen.

1.3 Füllen Sie die folgende Matrix aus, indem Sie die Werte von $OPT(i, j)$ per Hand ausrechnen. Entnehmen Sie aus der Matrix, welches die (bzw. eine) Palindrom-Teilfolge maximaler Länge ist.

	T	A	G	A	G	T	G
T							
A							
G							
A							
G							
T							
G							

Lösung

	T	A	G	A	G	T	G
T	1	1	1	3	3	5	5
A		1	1	3	3	3	3
G			1	1	3	3	3
A				1	1	1	3
G					1	1	3
T						1	1
G							1

leer steht für 0.

Länge des längsten Palindroms einer Teilfolge ist 5.

Eines dieser längsten Palindrome ist *TGAGT*

- 1.4 Der rekursive Ansatz, der in den vorherigen Aufgaben genutzt wurde, folgt dem *top-down*-Ansatz. Nutzen Sie nun den *bottom-up*-Ansatz, den Sie mit Schleifen implementieren können. Führen Sie für diesen Ansatz wieder eine Handsimulation durch und füllen Sie dazu wieder die Tabelle aus. Welche Werte werden dabei zuerst in die Tabelle geschrieben.

Lösung

Siehe Matrix: Zuerst die Diagonale

- 1.5 Was ist die Laufzeit einer Implementation mit Hilfe der OPT Funktion und dynamischer Programmierung.

Lösung

$O(n^2)$

Aufgabe 2: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Aufgabe 3: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Was Sie nach diesem Blatt wissen sollten:

- Was die Grundprinzipien der dynamischen Programmierung sind
- Welche beiden Voraussetzungen ein Problem erfüllen muss, damit dynamische Programmierung erfolgreich angewendet werden kann?
- Welche beiden Varianten zur Speicherung der Lösungswerte verwendet werden
- Wie man für ein gegebenes Problem eine OPT-Funktion herleiten kann